MIT 8103 Advanced Database Systems

Airline Reservation System · Ndukeobong Eton · Student ID 301926659

Case study

My case study is Airline Reservation System. I designed the database for passengers viewing and reservations, airline staff managing bookings and reviewing flight bookings, with separate tables named flight, passenger and reservation. It supports these operations: create a reservation for a passenger on a flight; confirm or cancel a reservation; review confirmed reservations by flight and date.

Portfolio 1: Database Design and Modelling

The goal is to represent three things: flights, passengers, and reservations. The design keeps each kind of information in its own table, then connects them with IDs.

- flight stores one scheduled flight: its number, departure date, route, and seat capacity.
- passenger stores a passenger's name and email.
- reservation records a passenger's booking for a flight, including when it was booked and its status The relational design is organised into 3 tables. Primary keys identify records, foreign keys preserve the links, and enforce the stated business rules.
- flight capacity must be greater than 0
- flight number and departure date identifies one flight
- passenger email must be unique reservation status: (pending, confirmed, cancelled)
- a confirmed seat on a flight cannot be used twice

The normalisation examples show the sample data at 1NF, 2NF and 3NF. All 11 deliberately invalid inserts were rejected without changing the valid sample rows.

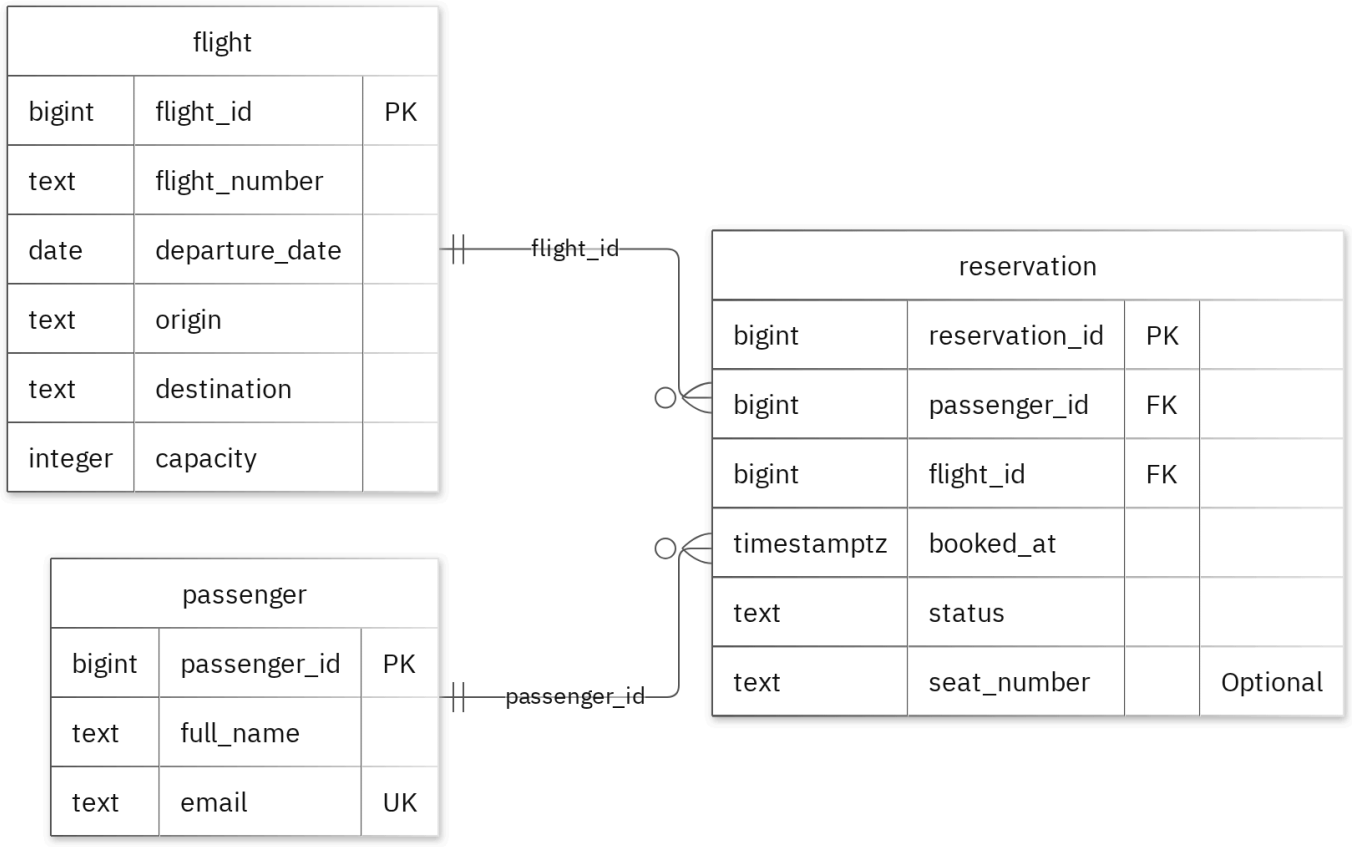


Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

I ran three SQL queries against the sample records. The join returned 2 rows and included Avery Stone with SW101. For the index comparison, I used 20,000 separate performance rows. The median lookup time changed from 1.145 ms (non-indexed) to 0.172 ms (indexed). Both phases returned the same records. The measured plan used the new index. These temporal data applies to this lookup and environment.

Portfolio 3: Transactions and Concurrency

I used a temporary counter on reservation to test COMMIT, ROLLBACK and READ COMMITTED isolation. The uncommitted update remained hidden from the other session, and ROLLBACK restored the earlier committed value. Without locking, two increments left the counter at 2. With SELECT FOR UPDATE, Session B waited for Session A and the final value was 3. The helper is confined to this exercise; the case's original business fields stay unchanged.

Portfolio 4: NoSQL and Advanced Data Models

I used reservation_events for the case's event documents. MongoDB inserted 4 sample events and returned the expected results for 4 queries. Validation rejected an invalid date type and a duplicate event ID. Embedded snapshots keep related details with an event, while the details field can differ by event type. These are fictional events; PostgreSQL continues to enforce its relational rules.

Repository and evidence

https://github.com/neton6659-afk/Eton_Ndukeobong_301926659

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots.

Portfolio 5: Distributed and Cloud Database Exercise

I proposed three MongoDB replica members across separate failure zones. The discussion covers primary elections, majority acknowledgement, secondary lag and sharding trade-offs for reservation_events. Replication supports availability, while backups address recovery. PostgreSQL needs its own availability plan. This is the permitted discussion option; no live replica set or failover test was deployed.

Individual reflection

This project significantly improved my understanding of database systems and how their different components interact to support real-world operations. Through the design process, I learned the importance of data structuring into related tables for better organisation as well as redundancy reduction. A better appreciation of how keys and integrity constraints helps ensure the accuracy and consistency of stored data.

In addition, a practical knowledge of query optimisation, indexing, and transaction management was attained, all of which play a vital role in the overall performance and reliability of a database system. Exploring MongoDB and distributed database architectures further expanded my understanding of modern data management approaches beyond the traditional relational model.

Overall, this project provided valuable insight into the challenges and considerations involved in designing, querying, optimising, and managing databases. The most important lesson I took away is that an effective database system depends on the successful integration of good design principles,data consistency,efficient performance management,and resilience to support long-term reliability.

AI-use declaration

I used GitHub Copilot and the portfolio generator to help prepare the database model, scripts and explanations. The generator ran the PostgreSQL and MongoDB exercises, checked their outputs and captured the result images. I remain responsible for checking and explaining the submitted work.

References

- PostgreSQL: constraints
- PostgreSQL: EXPLAIN
- PostgreSQL: isolation
- PostgreSQL: row locks
- MongoDB: data modelling
- MongoDB: replication
- MongoDB: sharding
- Mermaid: ER diagrams
- W3schools: SQL